



Algorithms Problem Solving

Level 2

My Round



Algorithms Problem Solving

Level 2

My Round

Definition

- ◆ This program calculates the rounded value of a decimal number without using the built-in `round()` function.
- ◆ The custom function `MyRound()` separates the integer part from the fractional part.
- ◆ If the fractional part is `0.5` or greater in absolute value, the integer part is adjusted toward the nearest integer.
- ◆ The program compares the custom result with C++'s built-in `round()` function.

Explanation

- ◆ ``ReadNumber()`` reads a floating-point number from the user.
- ◆ ``GetFractionPart()`` gets the fractional part by subtracting the integer part from the original number.
- ◆ For example:
$$33.54323 - 33 = 0.54323$$
- ◆ ``MyRound()`` first stores the integer part:
`int integerPart = number;`
- ◆ Since ``integerPart`` is an ``int``, the decimal part is removed.
- ◆ It then gets the fractional part using ``GetFractionPart()``.
- ◆ The condition:
`if (abs(FractionPart) >= 0.5)`
checks whether the fractional part is at least ``0.5`` in absolute value.
- ◆ For positive numbers, the integer part is increased by ``1``.
- ◆ For negative numbers, the integer part is decreased by ``1``.
- ◆ If the fractional part is less than ``0.5``, the integer part is returned unchanged.

Steps in main()

1. Ask the user to enter a decimal number.
2. Store the integer part of the number.
3. Calculate the fractional part.
4. Check whether the absolute fractional part is `>= 0.5`.
5. If the number is positive, increase the integer part by `1`.
6. If the number is negative, decrease the integer part by `1`.
7. If the fractional part is less than `0.5`, return the integer part unchanged.
8. Print the result from `MyRound()`.
9. Compare it with C++'s built-in `round()` function.

💡 Example

◆ Input:

33.54323

◆ Integer part:

33

◆ Fractional part:

$33.54323 - 33 = 0.54323$

◆ Since:

$0.54323 \geq 0.5$

◆ The number is positive, so the integer part is increased:

$33 + 1 = 34$

◆ Therefore, the rounded result is `34`.

➡ Result

My round Result: 34

C++ round Result: 34

✈ Note

- ◆ Important function:

```
float GetFractionPart(float number)
{
    return number - int(number);
}
```

- ◆ For positive numbers such as `33.54323`, this gives approximately `0.54323`.
- ◆ For negative numbers such as `-33.54323`, the integer part is `-33`, so the fractional part becomes approximately `-0.54323`.
- ◆ `abs(FractionPart)` allows the program to check the size of the fractional part regardless of its sign.
- ◆ The code uses `abs()` for the fractional part, but it does not use the built-in `round()` function.

⊘ Warning

- ◆ ``float`` has limited precision, so calculations involving decimal values may have small precision differences.
- ◆ ``GetFractionPart()`` truncates toward zero because ``int(number)`` removes the fractional part.
- ◆ The program assumes normal floating-point input.
- ◆ ``corecrt_math_defines.h`` is not needed for this program.
- ◆ ``cmath`` is needed because the code uses ``abs()`` and ``round()`` for comparison.

Comparison

`MyRound()`

- ✓ User-defined function.
- ✓ Does not use the built-in `round()` function.
- ✓ Separates the integer and fractional parts.
- ✓ Adjusts the integer part based on the fractional part.

`round()`

- ✓ Built-in C++ function.
- ✓ Directly rounds the number.
- ✓ Used in this program only to compare the result with `MyRound()`.

✦✦ Summary

- ✓ Read a decimal number.
- ✓ Extract the integer part.
- ✓ Extract the fractional part.
- ✓ Check whether the fractional part is at least `0.5` in absolute value.
- ✓ Increase or decrease the integer part accordingly.
- ✓ Return the rounded result.
- ✓ Compare the custom result with C++'s `round()` function.

Thank you 😊